

Member ID: _____

Time: _____

Rank: _____



JAVA PROGRAMMING

(340)

STATE 2022

Production:

Tollway Customer Database State _____ (425 points)

TOTAL POINTS _____ (*425 points*)

Test Time: 90 minutes

GENERAL GUIDELINES:

Failure to adhere to any of the following rules will result in disqualification:

1. Member must hand in this test booklet and all printouts if any. Failure to do so will result in disqualification.
2. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests (handwritten, photocopied, or keyed) are allowed in the testing area.
3. Electronic devices will be monitored according to ACT standards.

You will have ninety (90) minutes to complete your work.

Your name and/or school name should *not* appear on work you submit for grading.

1. Create a folder on the flash drive provided using your contestant number as the name of the folder.
2. Copy your entire solution/project into this folder.
3. Submit your entire solution/project so that the graders may open your project to review the source code.
4. Ensure that the files required to run your program are present and will execute on the flash drive provided.

*Note that the flash drive letter may *not* be the same when the program is graded as it was when you created the program.

*It is recommended that you use relative paths rather than absolute paths to ensure that the program will run regardless of the flash drive letter.

The graders will *not* compile or alter your source code to correct for this.
Submissions that do *not* contain source code will *not* be graded.

Assumptions to make when taking this assessment:

- The input file will contain only ASCII characters.
- A test input file will be available and named, "Names.txt."
- A starter file for the main class has been provided ("TollwayCustomerDataBaseState.java")

Development Standards:

- Your Code must use a consistent variable naming convention.
- All subroutines, functions, and methods must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any).
- If you create a class, then you must use Javadoc comments.

Tollway Text Import

You are being tasked with creating a software program that can import a text file, read the file, and assign its information to specific variables to construct objects (Name, CarInfo, Payment) that will be stored in another object (Customer), that will be stored in a List data structure. The program will print out the entire customer list once complete. You must use the provided “TollwayCustomerDataBaseState.java” source code template, and “Names.txt” files that have been provided.

Classes: Customer, Name, CarInfo & Payment

First, you will create these four classes that will serve to create each record for the customer list. The Customer constructor should be created in the following pattern: **Customer(Name, CarInfo, Payment)**. Each one of these classes will need an appropriate constructor for its respected fields. And each class will only have one method, which is a getter method that returns a string that will be used for printing the records to the console: **String getInfo()**. NOTE: the Payment object is required to format the deposit into US currency, including “\$” and “,” as well as the cents.

Main Driver Class:

TollwayCustomerDataBaseState

This class will host the main () method that will be used to run the program. Listed below are required methods you must create and use in your program. You are allowed to create other methods, however they will be required to have commenting (see the rubric).

Methods

- **BufferedReader getReader(String name):** this method imports the file properly for the BufferedReader. You need to catch exceptions in this method if a file is missing or some other type of file reading error.
- **Customer readCustomer(BufferedReader in):** this method will take information from the Names.txt file and store the data for the appropriate field into the appropriate object to create the customer record. You need to catch exceptions in this method if there is a file reading error during input.

Input:

The input to this problem consists of a text file (Names.txt). The text file contains a sequence of values to allow the program to create an accurate set of customer records to contain in a list. Each field is separated by commas. The entire text file is complete for all fields for all records. The information in the text file is stored in the following sequence:

LastName, FirstName, Make, Model, LicensePlate, Deposit, CreditCardNumber.

Input/Output Samples:

The output will be created in the following format

Example input from the text file:

Pedro, Smith, Toyota, Camry, DFG-456, 78.40, BPAFHJF0483897
Mary, Taylor, Toyota, Corolla, DFGF-098, 39.31, BPA4223442897

Example output to the console. You ARE required to create this format...each printed line will be formatted with the method **String getInfo()**. This method is required to be part of the **Customer, Name, CarInfo, and Payment** classes. There should be a total of 12 records.

Sample format convention with NO data (you will not see this print)

Name: FIRSTNAME LASTNAME
Car Information: MAKE MODEL LICENSE-PLATE#
Payment: \$DEPOSIT CREDITCARD#

Sample format with data

Name: Henry Ichiban
Car Information: Tesla X DGF-4356
Payment: \$3,445.67 BPA5678934456

Requirements:

1. You will completing an application with the main class named TollwayCustomerDataBaseState. The java starter file with the previous main class name has been provided.
2. You will create a helper classes named Customer, Name, CarInfo, and Payment
3. The helper classes Customer, Name, CarInfo, and Payment must have a method called **String getInfo()** that will be used to print all of the records based upon the output sample provided.
4. You must add the text file "Names.txt" and "TollwayCustomerDataBaseState.java" to your project folder for this application (it will not operate correctly without these two files).
5. Your contestant number must appear as a comment at the top of the main source code file.
6. If the input file is not found, then the program should display an appropriate message and exit.
7. The program will import and read the text file, storing the results into the appropriate objects that will be stored in a Customer object.
8. You must implement methods in TollwayCustomerDataBaseState:
 - a. **BufferedReader getReader(String name)**
 - b. **Customer readCustomer(BufferedReader in)**
9. You must create TollwayCustomerDataBaseState **main()** method that performs the following:
 - a. Call method **BufferedReader getReader(String name)** to access the text file.
 - b. Call method **readCustomer(BufferedReader in)** to read the text file and assign the appropriate value to the correct variables.
 - c. Use the variables to create customer objects that are added to a List.
 - d. After finished reading the text file: print the entire customer list by calling the appropriate method to print.
10. If properly executed, the program will display the output like the example above for all of the values in the text file.

Your application will be graded on the following criteria:		
Solution and Project		
The project is present on the flash drive	_____	10 points
The projects main class is named TollwayCustomerDataBaseState	_____	10 points
The projects helper classes are named Customer, Name, CarInfo, Payment (5 Points each)	_____	20 points
Program Execution		
The program runs from the USB flash drive	_____	15 points
If the program does <i>not</i> execute, then the remaining items in this section receive a score of zero.		
The TollwayCustomerDataBaseState class runs and reads the input file	_____	30 points
The TollwayCustomerDataBaseState class displays an error message if the file cannot be found	_____	20 points
The TollwayCustomerDataBaseState class stores data fields from Names.txt into proper variables	_____	20 points
The TollwayCustomerDataBaseState class stops reading the input file when <u>there is no more text</u>	_____	20 points
Program displays the correct output in the correct order for the records and the field with the formatting from the helper classes.	_____	30 points
The output should count and display 12 created records	_____	20 points
The deposit value was formatted for US currency including “\$” and “,” as well as the cents.	_____	10 Points
Source Code Review		
The source code is properly commented		
A comment containing the contestant number is present	_____	10 points
Methods and code sections are commented	_____	20 points
The TollwayCustomerDataBaseState class creates Customer objects from data contained in Names.txt	_____	30 points
Each Customer constructor is comprised of parameters from instantiated objects: Name, CarInfo, and Payment	_____	30 points
Name, CarInfo, and Payment objects correctly receive their information from the Names.txt file	_____	20 points
BufferedReader getReader(String name): Code uses try... catch for exception handling in TollwayCustomerDataBaseState class when reading the file	_____	20 points

Customer readCustomer(BufferedReader in): Code uses try... catch for exception handling in TollwayCustomerDataBaseState class when inputting from the file	_____	20 points
The TollwayCustomerDataBaseState class creates a List of Customers	_____	30 points
String getInfo(): this method is used by all instantiated objects for printing to the console	_____	20 points
The deposit value was formatted in the Payment object	_____	10 Points
Code uses a consistent variable naming convention	_____	10 points

Total Points = _____ / 425 points